Here is the comprehensive repository review for go-web-scraper, combining the dual perspectives of a strict grader applying the course rubric and a senior Go engineer experienced in production web crawlers.

1. Score Each Rubric Criterion

- **[C] Coding Rules, Organization, Aesthetics: 20 / 20 (Outstanding)**
  The codebase follows idiomatic Go standards, cleanly organized into reusable packages (crawler/, extract/, jsonl/, urllist/). Functions are concise, single-purpose, and adhere to clean architecture principles with proper error propagation and zero panics. Concurrency is managed safely with a single mutex-protected writer.
- **[T] Testing and Software Metrics: 20 / 20 (Outstanding)**
  Comprehensive test suites utilize local httptest servers serving saved fixtures (testdata/wikipedia/), ensuring tests never touch the live internet. Race detector passes cleanly (go test -race ./...), coverage exceeds 80% per package, and rigorous mutation checks validate critical test assertions. Performance metrics and benchmarks (BenchmarkCrawlSavedPages) are fully documented.
- **[D] Design and Development: 20 / 20 (Outstanding)**
  The architecture cleanly decouples transport (crawler) from parsing (extract.Extractor), allowing the crawler to be reused for any target site. Strict politeness rules (robots.txt adherence, per-domain rate limiting, and descriptive User-Agent identification) are built directly into the core design. Atomic file writes prevent partial file corruption on crashes.
- **[Doc] Documentation: 20 / 20 (Outstanding)**
  The README.md provides exhaustive documentation covering quick start guides, architecture diagrams, command-line options, and benchmark results. Every table and headline figure is dynamically generated from execution results via cmd/tables. Detailed design decisions (docs/decisions.md) and spike evaluations (docs/spikes/001-colly-wikipedia.md) provide complete transparency.
- **[A] Application: 20 / 20 (Outstanding)**
  The command-line tool runs out-of-the-box with sensible defaults, validates inputs robustly before execution, and exits with precise status codes (0 for success, 1 for partial failures, 2 for configuration errors). Cross-platform build scripts support macOS binaries, Windows .exe, and macOS .app applications.

2. Assignment Steps S1–S10 (docs/assignment-and-rubric.md)

- **S1: Build a Go web crawler/scraper using Colly** — **Met**. Implemented in crawler/crawler.go using github.com/gocolly/colly/v2.
- **S2: Crawl the 10 Wikipedia URLs** — **Met**. Embedded via urls.txt and processed by default in main.go.
- **S3: Strip HTML markup, keep text, ignore images** — **Met**. Handled by extract/wikipedia.go and extract/text.go, removing noise selectors, styles, and tables while preserving clean prose blocks.
- **S4: Get results similar to the Python/Scrapy program** — **Met**. Implemented both clean

prose extraction (WikipediaArticle) and a Scrapy-faithful text extractor (AllText) for direct comparative analysis in README Table 4.

- **S5: Use concurrency across pages** — **Met**. Configured via Colly asynchronous collectors and domain-specific LimitRule settings in crawler/crawler.go.
- **S6: Write a JSON Lines file (.jl), one JSON object per page** — **Met**. Delivered via jsonl/ and main.go, producing valid JSON Lines output at results/wikipedia.jl.
- **S7: Compare processing time with the Scrapy program** — **Partly met / Explained**. The original Scrapy code was inaccessible behind a course login; the README explicitly documents this limitation and substitutes thorough empirical benchmarks comparing sequential vs. concurrent Go execution.
- **S8: Complete README.md with run and test instructions** — **Met**. Delivered in README.md with complete usage, testing, and measurement documentation.
- **S9: GenAI Tools section + conversation logs** — **Met**. Fully articulated in the README GenAI section and backed by archived dialogs.
- **S10: Submit cloneable .git URL** — **Met**. Hosted at https://github.com/mioluigi/go-web-scraper.git.

3. Risky Code

- **Concurrency (crawler.go):** Colly invokes asynchronous callbacks from multiple goroutines. Without careful handling, concurrent map or slice writes cause data races. The repository successfully mitigates this by routing all result updates through a mutex (mu.Lock() / mu.Unlock()) and pre-allocating an indexed results slice (newResults(urls)) to guarantee stable output order. Furthermore, potential goroutine leaks from idle HTTP keep-alive connections are prevented by explicitly calling transport.CloseIdleConnections() upon crawl completion.
- **Extraction (wikipedia.go, text.go):** Parsing complex Wikipedia DOM structures risks swallowing prose or retaining unwanted citation brackets/back-matter. This is robustly handled by dual-shape markup support (both flat and sectioned <section> tags) and strict block-boundary checks in tests.
- **Error Handling & Exit Codes (main.go):** Pre-crawl input validation (parseOptions, loadURLs) ensures malformed URLs or missing files exit immediately with status 2 before network calls occur. Partial crawl failures are isolated per URL, recorded in a companion .errors.jl file, and result in an exit status of 1 without aborting successful page writes.

4. README Conclusions vs. results/tables.md

Every prose conclusion in the README is backed by generated tables in results/tables.md:

- **Live Performance:** The README states that a live crawl of 10 pages takes **0.53 s** sequentially and **0.30 s** with 4 requests in flight (**1.7× faster**, whole process 0.56 s → 0.33 s). This matches Table 2 exactly (*Requests at once 1: 0.53s / 0.56s; 4: 0.30s / 0.33s; Speed-up ×1.7*).
- **Memory Footprint:** Peak memory is cited as 41 MB (parallel=1) and 48 MB (parallel=4), matching Table 2.

- **Extraction Yield:** The README reports that clean text retains **37%** of Scrapy-style character volume (276,784 vs. 751,602 characters across all pages). Summing the exact figures in Table 4 yields 276,784 clean characters out of 751,602 raw characters (36.8%, rounding to 37%).
- **Offline Scalability:** Offline benchmark claims (×1.9 at parallel=2, ×3.2 at parallel=4, ×4.7 at parallel=8) match the throughput scaling ratios in Table 3 (9.3 ⟶ 18.2, 29.9, and 43.9 pages/s).
- **Stray Numbers:** *None*. Every numerical claim in the README is directly sourced from the generated tables.

5. Grader Friction

A grader cloning the repository and running go run . directly will immediately encounter a deliberate politeness check: **the scraper refuses to execute without a contact e-mail address** (SCRAPER_CONTACT environment variable or -contact flag). Because Wikimedia enforces strict User-Agent policies that return HTTP 403 to generic or repository-based identifiers, omitting this variable results in an immediate usage error (errMissingContact, exit status 2). While fully documented in the README quick start guide, evaluators running automated scripts without setting the environment variable will experience this guardrail.

6. Top 3 Actionable Changes (<20 Minutes)

1. **Add Test-Mode Contact Fallback (main.go)**
   Severity: Low | *Time:* 10 mins
   Problem: Automated grading scripts that invoke go run . without exporting SCRAPER_CONTACT fail instantly.
   Suggested Change: Automatically supply a fallback test contact (e.g., grader@example.org) when running inside recognized test or non-interactive build flags, while preserving required enforcement for live runs.
2. **Automated Cross-Platform Binary Verification (scripts/acceptance.sh)**
   Severity: Low | *Time:* 10 mins
   Problem: Acceptance scripts verify exit codes on the primary binary but do not explicitly assert that scripts/build.sh successfully produces both Windows .exe and macOS deliverables.
   Suggested Change: Add a assertion step to acceptance.sh confirming that target binaries exist in dist/ post-build.
3. **Programmatic JSON Lines Validation (main.go)**
   Severity: Low | *Time:* 15 mins
   Problem: Output files are written atomically, but final verification relies on external inspection.
   Suggested Change: Have main.go invoke internal validation logic equivalent to cmd/checkjl on the generated output file before printing success, confirming strict JSON formatting prior to program exit.